

What is SQL?

SQL stands for **Structured Query Language**. It is a standardized programming language used to **manage** and manipulate relational databases. It enables users to perform a variety of tasks such as querying data, creating and modifying database structures, and managing access permissions. SQL is widely used across various relational database management systems such as **MySQL**, **PostgreSQL**, Oracle, and SQL Server.

In this article, we will learn about what SQL is, and its **functionality**, structure, and practical uses. We will also learn its **characteristics**, rules, commands, etc from basic to advanced technical features.

What is SQL?

Data is at the core of every application, and **SQL** (Structured Query Language) manages and interacts with this data. Whether we're handling a small user database or analyzing **terabytes** of sales records, SQL allows efficient querying, **updating**, and **management** of relational databases.

When data needs to be retrieved from a database, SQL is used to construct and send the request. The Database Management System (DBMS) processes the **SQL query**, retrieves the requested data, and returns it to the user or application. Instead of specifying step-by-step procedures, SQL statements describe what data should be retrieved, **organized**, or modified, allowing the DBMS to handle how the operations are executed efficiently.

In common usage, SQL encompasses **DDL** and **DML** commands for [CREATE](#), [UPDATE](#), [MODIFY](#), or other operations on database structure.

SQL History

- SQL was invented in 1970s and was first commercially distributed by Oracle.
- The original name was given by IBM as Structured English Query Language, abbreviated by the acronym SEQUEL.

Components of a SQL System

A SQL system consists of several key components that work together to enable efficient data storage, **retrieval**, and manipulation. Understanding these components is crucial for mastering [SQL](#) and its role in relational database systems. Some of the Key components of a SQL System are:

- **Databases:** Databases are structured collections of data organized into tables, rows, and columns. [Databases](#) serve as repositories for storing information efficiently and provide a way to manage and access data.
- **Tables:** Tables are the fundamental building blocks of a database, consisting of rows (records) and columns (attributes or fields). Tables ensure data integrity and consistency by defining the structure and relationships of the stored information.
- **Queries:** Queries are SQL commands used to interact with databases. They enable users to retrieve, update, insert, or delete data from tables, allowing for efficient data manipulation and retrieval.
- **Constraints:** Constraints are rules applied to tables to maintain data integrity. [Constraints](#) define conditions that data must meet to be stored in the database, ensuring accuracy and consistency.
- **Stored Procedures:** Stored procedures are pre-compiled SQL statements stored in the database. [Stored procedures](#) can accept parameters, execute complex operations, and return results, enhancing efficiency, reusability, and security in database management.
- **Transactions:** Transactions are groups of SQL statements that are executed as a single unit of work. [Transactions](#) ensure data consistency and integrity by allowing for the rollback of changes if any part of the transaction fails.

Some other important components include:

- Data Types

- Indexes
- Views
- Security and Permissions
- Joins

SQL Injection is a cyberattack where malicious SQL queries are injected into input fields to manipulate a database, enabling unauthorized access, data modification, or corruption. Using parameterized queries and input validation helps prevent such attacks.

What are the Characteristics of SQL?

- **User-Friendly and Accessible:** SQL is designed for a broad range of users, including those with minimal programming experience, making it approachable for non-technical individuals.
- **Declarative Language:** As a non-procedural language, SQL allows users to specify what data is needed rather than how to retrieve it, focusing on the desired results rather than the retrieval process.
- **Efficient Database Management:** SQL enables the creation, modification, and management of databases efficiently, saving time and simplifying complex database operations.
- **Standardized Language:** Based on ANSI (American National Standards Institute) and ISO (International Organization for Standardization) standards, SQL ensures consistency and stability across various database management systems (DBMS).
- **Command Structure:** SQL does not require a continuation character for multi-line queries, allowing flexibility in writing commands across one or multiple lines.
- **Execution Mechanism:** Queries are executed using a termination character (e.g., a semicolon ;), enabling immediate and accurate command processing.
- **Built-in Functionality:** SQL includes a rich set of built-in functions for data manipulation, aggregation, and formatting, empowering users to handle diverse data-processing needs effectively.

How SQL Works?

Structured Query Language (SQL) operates on a server machine, where it processes database queries and returns results efficiently. Below are the key software components involved in the SQL execution process.

1. **Input:** The process begins when a user submits an SQL query through a database interface or application. This query typically specifies the desired operation, such as data retrieval, insertion, updating, or deletion.
2. **Parsing:** The query is passed to the query processor, which breaks it into smaller units called tokens. These tokens represent keywords, table names, column names, and other elements of the query. The processor then validates the syntax against SQL standards and the database schema to ensure the query is well-formed and executable.
3. **Optimization:** After parsing, the query is handed to the optimizer, which evaluates multiple ways to execute the query. The optimizer considers factors like indexes, table statistics, and available resources to generate the most efficient execution plan. This step ensures that the query runs with minimal resource consumption and maximum performance.
4. **Execution:** The execution engine follows the plan provided by the optimizer. It interacts with the storage engine, which retrieves, manipulates, or updates the required data from the database tables. During this step, SQL statements like SELECT, INSERT, UPDATE, or DELETE are translated into actions performed on the underlying data.
5. **Output:** Once the execution engine processes the query, the result is formatted and returned to the user. Depending on the query type, the output could be a result set (for SELECT queries) or an acknowledgment of the operation (for INSERT, UPDATE, or DELETE queries).

By combining these steps, SQL ensures the seamless interaction between users and relational databases, enabling efficient data manipulation and retrieval.

Rules for Writing SQL Queries

There are certain rules for SQL which would ensure consistency and functionality across databases. By following these rules, queries will be well formed and well executed in any database.

- **Statement Termination:** Every SQL statement ends with a semicolon (;), signaling the DBMS to execute the command.
- **Case Insensitivity:** SQL keywords (e.g., SELECT, INSERT) are case-insensitive, but database names and column names may be case-sensitive depending on the DBMS.
- **Whitespace Flexibility:** SQL statements can span multiple lines, but keywords and identifiers must be separated by at least one space.
- **Unique Identifiers:** Reserved words (e.g., SELECT, FROM) cannot be used as table or column names unless enclosed in double quotes (") or backticks (`), depending on the DBMS.
- **Comments:** Comments enhance readability:
 - Single-line comments: —
 - Multi-line comments: /* ... */
- **Data Integrity:** Constraints like NOT NULL, UNIQUE, and PRIMARY KEY must be defined correctly to maintain data consistency.
- **String Literals:** String values must be enclosed in single quotes (').
- **Valid Identifiers:** Table and column names must:
 - Begin with an alphabetic character.
 - Contain up to 30 characters.
 - Avoid special characters except underscores (_).

By following these rules, SQL users ensure reliable query execution and maintainable database structures.

What are SQL Commands?

Structured Query Language (SQL) commands are standardized instructions used by developers to interact with data stored in **relational databases**. These commands allow for the **creation**, manipulation, retrieval, and **control of data**, as well as database structures. SQL commands are categorized based on their specific functionalities:

1. Data Definition Language

These commands are used to **define** the structure of database objects by **creating**, **altering**, and **dropping** the database objects. Based on the needs of the business, database engineers create and modify database objects using DDL. The [**CREATE**](#) command, for instance, is used by the database engineer to create database objects like tables, views, and indexes.

Command	Description
CREATE	Creates a new table, a view on a table, or some other object in the database.
ALTER	Modifies an existing database object, such as a table
DROP	Deletes an entire table, a view of a table, or other objects in the database

2. Data Manipulation Language

A relational database can be updated with new data using data manipulation language ([DML](#)) statements. The [INSERT](#) command, for instance, is used by an application to add a new record to the database.

Command	Description
INSERT	Creates a record.
UPDATE	Modifies records.
DELETE	Deletes records.

3. Data Query Language

Data retrieval instructions are written in the data query language ([DQL](#)), which is used to access relational databases. The [SELECT](#) command is used by software programs to filter and return particular results from a SQL table.

4. Data Control language

DCL commands manage user access to the database by granting or revoking permissions. Database administrators use [DCL](#) to enforce security and control access to database objects.

Command	Description
GRANT	Gives a privilege to the user.
REVOKE	Takes back privileges granted by the user.

5. Transaction Control Language

TCL commands manage transactions in relational databases, ensuring data integrity and consistency. These commands are used to commit changes or roll back operations in case of errors.

Command	Description
COMMIT	Saves all changes made during the current transaction on a permanent basis. Some databases provide an auto-commit feature, which can be configured using settings.
ROLLBACK	Reverts changes made during the current transaction, ensuring no unwanted changes are saved.
SAVEPOINT	Sets a point within a transaction to which changes can be rolled back, allowing partial rollbacks

Benefits of SQL

- **Efficiency:** SQL is designed to handle complex queries and large datasets with optimal performance, making data retrieval and manipulation seamless.
- **Standardization:** As an ANSI and ISO standard language, SQL provides a universal method to interact with relational databases across platforms.
- **Scalability:** SQL supports databases ranging from small-scale applications to enterprise-level systems, ensuring smooth operations regardless of size.
- **Flexibility:** SQL can be extended with procedural programming (e.g., PL/SQL, T-SQL) to build complex business logic and custom functions.

Limitations of SQL

- **Complexity in Advanced Operations:** Advanced functionalities such as indexing, query optimization, and performance tuning require in-depth technical knowledge.
- **Scalability Concerns:** SQL performs best with structured data; handling unstructured data or massive distributed systems can pose challenges.
- **Platform-Specific Variations:** While SQL is standardized, many databases implement unique extensions, leading to portability and compatibility issues.

SQL Use Cases

- **E-Commerce:** Manage customer orders, product catalogs, and inventory.
- **Healthcare:** Maintain patient records and appointment schedules.
- **Banking:** Analyze transaction histories and generate financial reports.
- **Web Development:** Power dynamic websites with user-specific content.
- **Machine Learning and Data Science:** Combine SQL databases with tools like Python, R, and TensorFlow to streamline machine learning workflows.

SQL Data Types

SQL Data Types are very important in **relational databases**. It ensures that data is stored efficiently and accurately. Data types define the type of value a column can hold, such as **numbers**, text, or **dates**.

Understanding SQL Data Types is critical for database administrators, **developers**, and data analysts to design robust databases and optimize performance.

In this article, we will learn a comprehensive overview of SQL Data Types, their significance, and **practical examples** for various real-world scenarios. We will cover different SQL Data Types like **Numeric**, **Date and time**, Character, etc.

Why SQL Data Types Matter

SQL data types are essential for designing relational databases, as they determine how data is stored, managed, and interacted with. Choosing the right data type ensures:

- **Data Integrity:** Prevents invalid data from being entered into the database (e.g., storing text in a numeric column).
- **Efficient Storage:** Reduces storage costs by allocating only the necessary space for data.
- **Query Performance:** Improves query performance by enabling faster indexing and search operations.
- **Application Compatibility:** Ensures smooth interaction between the database and applications by enforcing consistency in data handling.

For example, using **DECIMAL** instead of **FLOAT** for financial calculations ensures precision and avoids rounding errors, making it critical in industries like finance and e-commerce.

What are SQL Data Types?

SQL developers must know what data type will be stored inside each column while creating a table. SQL data types guide how the system processes data and dictates the behavior of operations like sorting, searching, and calculations. The main SQL data types include:

- **Numeric Data Types**
- **Character and String Data Types**
- **Date and Time Data Types**
- **Binary Data Types**
- **Boolean Data Types**
- **Special Data Types**

Numeric Data Types

Numeric data types are fundamental to [database](#) design and are used to store numbers, whether they are integers, decimals, or floating-point numbers. These data types allow for mathematical operations like **addition**, **subtraction**, **multiplication**, and division, which makes them essential for managing financial, scientific, and analytical data.

Exact Numeric Datatype

Exact numeric types are used when precise numeric values are needed, such as for financial data, quantities, and counts. Some common exact numeric types include:

Data Type	From	To
BigInt	-263 (-9,223,372,036,854,775,808)	263 -1 (9,223,372,036,854,775,807)
Int	-231 (-2,147,483,648)	231-1 (2,147,483,647)
Smallint	-215 (-32,768)	215-1 (32,767)
Tinyint	0	28-1 (255)
Bit	0	1
Decimal	-1038+1	1038-1
Numeric	-1038+1	1038-1
Money	-922,337,203,685,477.5808	922,337,203,685,477.5807
SmallMoney	-214,748.3648	214,748.3647

Approximate Numeric Datatype

These types are used to store **approximate values**, such as scientific measurements or large ranges of data that don't need exact precision.

Data Type	From	To
Float	-1.79E+308	1.79E+308
Real	-3.40E+38	3.40E+38

Character and String Data Types

Character data types are used to store text or character-based data. These include:

Character String Data Types

Data Type	Description
Char	The maximum length of 8000 characters.(Fixed-Length non-Unicode Characters)
Varchar	The maximum length of 8000 characters.(Variable-Length non-Unicode Characters)
Varchar(max)	The maximum length of 231 characters(SQL Server 2005 only).(Variable Length non-Unicode data)
Text	The maximum length of 2,127,483,647 characters(Variable Length non-Unicode data)

Unicode Character String Data Types

Unicode data types are used to store characters from any language, supporting a wider variety of characters. These are given in below table.

Data Type	Description
Nchar	The maximum length of 4000 characters(Fixed-Length Unicode Characters)
Nvarchar	The maximum length of 4000 characters.(Variable-Length Unicode Characters)
Nvarchar(max)	The maximum length of 231 characters(SQL Server 2005 only).(Variable Length Unicode data)

Date and Time Data Type in SQL

SQL provides several data types for storing date and time information. These are given in the below table.

Data Type	Description
DATE	A data type is used to store the data of date in a record
TIME	A data type is used to store the data of time in a record
DATETIME	A data type is used to store both the data,date, and time in the record.

Binary Data Types in SQL

Binary data types are used to store binary data such as images, videos, or other file types. These include:

Data Type	Description	Max Length
Binary	Fixed-length binary data.	8000 bytes
VarBinary	Variable-length binary data.	8000 bytes
Image	Stores binary data as images.	2,147,483,647 bytes

Boolean Data Type in SQL

The **BOOLEAN** data types are used to store logical values, typically TRUE or FALSE.

Data Type	Description
BOOLEAN	Stores a logical value (TRUE/FALSE).

Special Data Types

XML Data Type

XML data type allows storage of XML documents and fragments in a SQL Server database

DataType	Description
XML Datatype	store data in the format of XML datatype

Spatial Data Type

A datatype is used for storing planar spatial data, such as points, lines, and polygons, in a database table.

DataType	Description
Geometry	stores planar spatial data, such as points, lines, and polygons, in a database table.

Array Datatype

SQL Server does not have a built-in array data type. However, it is possible to simulate arrays using tables or XML data types.

SQL Commands | DDL, DQL, DML, DCL and TCL Commands

SQL commands are essential for managing databases effectively. These commands are divided into categories such as Data Definition Language (**DDL**), Data Manipulation Language (**DML**), Data Control Language (**DCL**), Data Query Language (**DQL**), and Transaction Control Language (**TCL**).

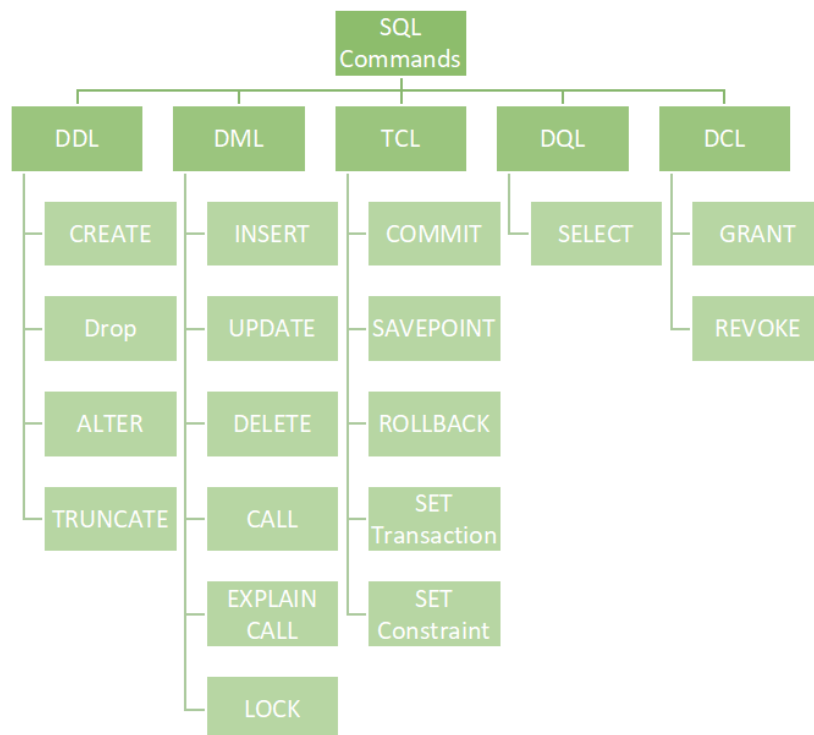
In this article, we will explain the different **types of SQL commands**, including **DDL**, **DML**, **DCL**, **DQL**, and **TCL**. These **SQL sublanguages** serve specific purposes and are important for effective **database management**.

What are SQL Commands?

SQL Commands are like instructions to a table. It is used to interact with the database with some operations. It is also used to perform specific tasks, functions, and queries of data. SQL can perform various tasks like creating a table, adding data to tables, dropping the table, modifying the table, set permission for users.

SQL Commands are mainly categorized into five categories:

- **DDL** – Data Definition Language
- **DQL** – Data Query Language
- **DML** – Data Manipulation Language
- **DCL** – Data Control Language
- **TCL** – Transaction Control Language



1. Data Definition Language (DDL) in SQL

DDL or Data Definition Language actually consists of the [SQL](#) commands that can be used to **defining, altering, and deleting** database structures such as **tables, indexes, and schemas**. It simply deals with descriptions of the database schema and is used to **create** and **modify** the structure of database objects in the database

Common DDL Commands

Command	Description	Syntax
CREATE	Create database or its objects (table, index, function, views, store procedure, and triggers)	CREATE TABLE table_name (column1 data_type, column2 data_type, ...);
DROP	Delete objects from the database	DROP TABLE table_name;
ALTER	Alter the structure of the database	ALTER TABLE table_name ADD COLUMN column_name data_type;
TRUNCATE	Remove all records from a table, including all spaces allocated for the records are removed	TRUNCATE TABLE table_name;
COMMENT	Add comments to the data dictionary	COMMENT 'comment_text' ON TABLE table_name;
RENAME	Rename an object existing in the database	RENAME TABLE old_table_name TO new_table_name;

Example of DDL

```
CREATE TABLE employees (  
    employee_id INT PRIMARY KEY,  
    first_name VARCHAR(50),  
    last_name VARCHAR(50),  
    hire_date DATE  
);
```

In this example, a new table called employees is created with columns for employee ID, first name, last name, and hire date.

2. Data Query Language (DQL) in SQL

DQL statements are used for performing queries on the data within **schema objects**. The purpose of the DQL Command is to get some **schema relation** based on the query passed to it. This command allows getting the data out of the database to perform operations with it. When a [SELECT](#) is fired against a table or tables the result is compiled into a further **temporary table**, which is displayed or perhaps received by the program.

DQL Command

Command	Description	Syntax
<u>SELECT</u>	It is used to retrieve data from the database	SELECT column1, column2, ...FROM table_name WHERE condition;

Example of DQL

```
SELECT first_name, last_name, hire_date
FROM employees
WHERE department = 'Sales'
ORDER BY hire_date DESC;
```

This query retrieves employees' first and last names, along with their hire dates, from the employees table, specifically for those in the 'Sales' department, sorted by hire date.

3. Data Manipulation Language (DML) in SQL

The **SQL commands** that deal with the **manipulation of data** present in the database belong to **DML** or Data Manipulation Language and this includes most of the **SQL statements**. It is the component of the SQL statement that controls access to data and to the [database](#). Basically, DCL statements are grouped with DML statements.

Common DML Commands

Command	Description	Syntax
<u>INSERT</u>	Insert data into a table	INSERT INTO table_name (column1, column2, ...) VALUES (value1, value2, ...);
<u>UPDATE</u>	Update existing data within a table	UPDATE table_name SET column1 = value1, column2 = value2 WHERE condition;
<u>DELETE</u>	Delete records from a database table	DELETE FROM table_name WHERE condition;
<u>LOCK</u>	Table control concurrency	LOCK TABLE table_name IN lock_mode;
CALL	Call a PL/SQL or JAVA subprogram	CALL procedure_name(arguments);
EXPLAIN PLAN	Describe the access path to data	EXPLAIN PLAN FOR SELECT * FROM table_name;

Example of DML

```
INSERT INTO employees (first_name, last_name, department)
VALUES ('Jane', 'Smith', 'HR');
```

This query inserts a new record into the employees table with the first name 'Jane', last name 'Smith', and department 'HR'.

4. Data Control Language (DCL) in SQL

DCL (Data Control Language) includes commands such as **GRANT** and **REVOKE** which mainly deal with the **rights, permissions**, and other controls of the **database system**. These commands are used to **control access** to data in the database by **granting or revoking permissions**.

Common DCL Commands

Command	Description	Syntax
GRANT	Assigns new privileges to a user account, allowing access to specific database objects, actions, or functions.	GRANT privilege_type [(column_list)] ON [object_type] object_name TO user [WITH GRANT OPTION];
REVOKE	Removes previously granted privileges from a user account, taking away their access to certain database objects or actions.	REVOKE [GRANT OPTION FOR] privilege_type [(column_list)] ON [object_type] object_name FROM user [CASCADE];

Example of DCL

```
GRANT SELECT, UPDATE ON employees TO user_name;
```

This command grants the user user_name the permissions to select and update records in the employees table.

5.Transaction Control Language (TCL) in SQL

Transactions group a set of tasks into a **single execution unit**. Each transaction begins with a specific task and ends when all the tasks in the group are successfully completed. If any of the **tasks fail**, the transaction fails. Therefore, a transaction has only two results: **success** or **failure**. We can explore more about **transactions** [here](#).

Common TCL Commands

Command	Description	Syntax
BEGIN TRANSACTION	Starts a new transaction	BEGIN TRANSACTION [transaction_name];
COMMIT	Saves all changes made during the transaction	COMMIT;
ROLLBACK	Undoes all changes made during the transaction	ROLLBACK;

Command	Description	Syntax
SAVEPOINT	Creates a savepoint within the current transaction	SAVEPOINT savepoint_name;

Example of TCL

```
BEGIN TRANSACTION;
UPDATE employees SET department = 'Marketing' WHERE department = 'Sales';
SAVEPOINT before_update;
UPDATE employees SET department = 'IT' WHERE department = 'HR';
ROLLBACK TO SAVEPOINT before_update;
COMMIT;
```

In this example, a transaction is started, changes are made, and a savepoint is set. If needed, the transaction can be rolled back to the savepoint before being committed.

Important SQL Commands

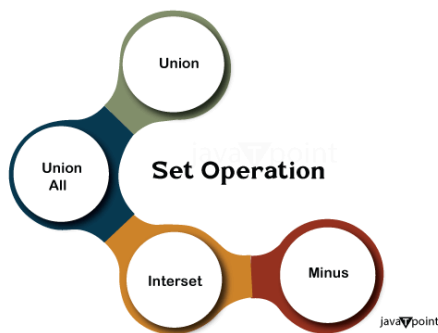
1. **SELECT:** Used to retrieve data from a database.
2. **INSERT:** Used to add new data to a database.
3. **UPDATE:** Used to modify existing data in a database.
4. **DELETE:** Used to remove data from a database.
5. **CREATE TABLE:** Used to create a new table in a database.
6. **ALTER TABLE:** Used to modify the structure of an existing table.
7. **DROP TABLE:** Used to delete an entire table from a database.
8. **WHERE:** Used to filter rows based on a specified condition.
9. **ORDER BY:** Used to sort the result set in ascending or descending order.
10. **JOIN:** Used to combine rows from two or more tables based on a related column between them.

SQL Set Operation

Whenever we need to combine the results from two or more SELECT statements. The SQL fulfill these requirements with the help of SET operations. The result of each select statement can be treated as a set and the SQL set operation can be applied to those sets to arrive at the result.

Types of Set Operation

1. Union
2. Union All
3. Intersect
4. Minus



The Syntax for using SET operator is:

1. SELECT statement1 <set operator> SELECT statement2;
2. [order by clause];

In the above syntax, **set operator** is the type of set operation you want to perform i.e. Union, Union all, Intersect, minus. **SELECT statement1** contains first table columns and **SELECT statement2** contains columns of second table.

SQL provides some rules to be followed when writing SQL queries:

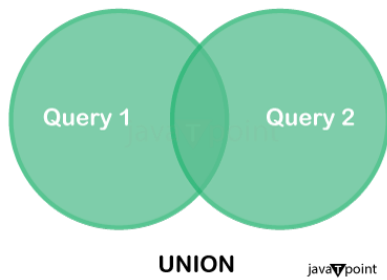
- In each of the individual queries that make up a compound query, the number of columns and data types of the select list columns is very similar. However the column names may be different.
- You can use the same or different set operators with two or more tables.
- In SQL statement all set operators have equal precedence if it contains more than one set operator then SQL evaluates them from left to right provided no parentheses are specified.
- You cannot specify an ORDER BY clause in any of the individual queries contained in a compound query. You can order full compound query results.
- You cannot use set operators on LOB data types like LOB, BLOB.
- It can be used in sub queries.

Let's explain each set operator one by one in detail with various examples.

1. Union

- The SQL Union operation is used to combine the result of two or more SQL SELECT queries.

- In the union operation, all the number of datatype and columns must be same in both the tables on which UNION operation is being applied.
- The union operation eliminates the duplicate rows from its resultset.



The above diagram shows the graphical representation of UNION set operator.

Syntax

1. SELECT column_name FROM table1
2. UNION
3. SELECT column_name FROM table2;

In the above syntax, column_name is the column name on which you want to perform the set operation, table1 is the name of the first table and table2 is the name of the second table and Union is the set operator.

Example:

The First table

1	Jack
2	Harry
3	Jackson

The Second table

3	Jackson
4	Stephan
5	David

Union SQL query will be:

1. SELECT * FROM First
2. UNION

3. `SELECT * FROM Second;`

The resultset table will look like:

1	Jack
2	Harry
3	Jackson
4	Stephan
5	David

Explanation: In the above example, first select statement will retrieve all the values from the table and the second select statement will retrieve all the values from the table. On using UNION set operator, all the rows retrieved from above two queries will be combined and all duplicates are removed.

2. Union All

Union All operation is equal to the Union operation. It returns the set without removing duplication and sorting the data.

The above diagram shows the graphical representation of UNION ALL set operator.

Syntax:

1. `SELECT column_name FROM table1`
2. `UNION ALL`
3. `SELECT column_name FROM table2;`

In the above syntax, column_name is the column name on which you want to perform the set operation, table1 is the name of the first table and table2 is the name of the second table and Union All is the set operator.

Example: Using the above First and Second table.

Union All query will be like:

1. `SELECT * FROM First`
2. `UNION ALL`
3. `SELECT * FROM Second;`

The resultset table will look like:

1	Jack
2	Harry
3	Jackson
3	Jackson
4	Stephan
5	David

Explanation: In the example above, if you use the UNION ALL set operator, duplicate rows are not filtered out of the list. The output is stored in ascending order of the first column of the select clause excluding the union all. Also null values are not ignored during duplicate checking.

3. Intersect

- It is used to combine two SELECT statements. The Intersect operation returns the common rows from both the SELECT statements.
- In the Intersect operation, the number of datatype and columns must be the same.
- It has no duplicates and it arranges the data in ascending order by default.
- It does not ignore NULL values.
- By using intersection set operator, reversing the order of table does not alter the result.

The above diagram shows the graphical representation of INTERSECT set operator.

Syntax

1. SELECT column_name FROM table1
2. INTERSECT
3. SELECT column_name FROM table2;

In the above syntax, column_name is the column name on which you want to perform the set operation, table1 is the name of the first table and table2 is the name of the second table and Intersect is the set operator.

Example:

Using the above First and Second table.

Intersect query will be:

1. SELECT * FROM First
2. INTERSECT
3. SELECT * FROM Second;

The result set table will look like:

3	Jackson
---	---------

Explanation: In the above example, on execution the common rows retrieved by both the SELECT statement.

4. Minus

- It combines the result of two SELECT statements. Minus operator is used to display the rows which are present in the first query but absent in the second query.
- It has no duplicates and data arranged in ascending order by default.

The above diagram shows the graphical representation of MINUS set operator.

Syntax:

1. SELECT column_name FROM table1
2. MINUS
3. SELECT column_name FROM table2;

In the above syntax, column_name is the column name on which you want to perform the set operation, table1 is the name of the first table and table2 is the name of the second table and minus is the set operator.

Example

Using the above First and Second table.

Minus query will be:

1. SELECT * FROM First
2. MINUS
3. SELECT * FROM Second;

The resultset table will look like:

1	Jack
2	Harry

Explanation: In the above example, on execution all the rows retrieved by First SELECT statement except those rows retrieved from SELECT statement which are not common to first SELECT will be retrieved i.e. common rows in the second table are subtracted from the first table and remaining rows retrieved from the first SELECT statement are displayed.

SQL Aggregate functions

SQL Aggregate Functions are used to perform calculations on a set of rows and return a single value. These functions are particularly useful when we need to summarize, analyze, or group large datasets in **SQL databases**. Whether you're working with sales data, employee records, or product inventories, aggregate functions help us derive meaningful insights.

They are often used with the **GROUP BY** clause in SQL to summarize data for each group. Commonly used aggregate functions include **COUNT()**, **SUM()**, **AVG()**, **MIN()**, and **MAX()**. In this article, we will explain the most commonly used SQL aggregate functions, their syntax, practical examples, and how to make the best use of them in your queries

What Are SQL Aggregate Functions?

SQL Aggregate Functions are used to perform calculations on multiple rows of data and return a single summarized result. These functions are typically used with the **GROUP BY clause** to organize data into groups, where we can then perform calculations like **summing**, **averaging**, or **counting**. Aggregate functions allow you to make sense of large datasets by collapsing them into meaningful summaries.

Key Features of SQL Aggregate Functions:

- **Operate on groups of rows:** They work on a set of rows and return a single value.
- **Ignore NULLs:** Most aggregate functions ignore NULL values, except for `COUNT (*)`.
- **Used with GROUP BY:** To perform calculations on grouped data, you often use aggregate functions with `GROUP BY`.
- **Can be combined with other SQL clauses:** Aggregate functions can be used alongside `HAVING`, `ORDER BY`, and other SQL clauses to filter or sort results.

Commonly Used SQL Aggregate Functions

Below are the most frequently used aggregate functions in SQL.

1.Count()

The `COUNT ()` function returns the number of rows that match a given condition or are present in a column.

- `COUNT (*)`: Counts all rows.
- `COUNT(column_name)`: Counts non-NULL values in the specified column.
- `COUNT(DISTINCT column_name)`: Counts unique non-NULL values in the column.

Examples:

```
-- Total number of records in the table
SELECT COUNT(*) AS TotalRecords FROM Employee;

-- Count of non-NULL salaries
SELECT COUNT(Salary) AS NonNullSalaries FROM Employee;

-- Count of unique non-NULL salaries
SELECT COUNT(DISTINCT Salary) AS UniqueSalaries FROM Employee;
```

2. SUM()

The `SUM()` function calculates the total sum of a numeric column.

- `SUM(column_name)`: Returns the total sum of all non-NULL values in a column.

Examples:

```
-- Calculate the total salary
SELECT SUM(Salary) AS TotalSalary FROM Employee;

-- Calculate the sum of unique salaries
SELECT SUM(DISTINCT Salary) AS DistinctSalarySum FROM Employee;
```

3. AVG()

The `AVG()` function calculates the average of a numeric column. It divides the sum of the column by the number of non-NULL rows.

- `AVG(column_name)`: Returns the average of the non-NULL values in the column.

Examples:

```
-- Calculate the average salary
SELECT AVG(Salary) AS AverageSalary FROM Employee;

-- Average of distinct salaries
SELECT AVG(DISTINCT Salary) AS DistinctAvgSalary FROM Employee;
```

4. MIN() and MAX()

The `MIN()` and `MAX()` functions return the smallest and largest values, respectively, from a column.

- `MIN(column_name)`: Returns the minimum value.
- `MAX(column_name)`: Returns the maximum value.

Examples:

```
-- Find the highest salary
SELECT MAX(Salary) AS HighestSalary FROM Employee;

-- Find the lowest salary
SELECT MIN(Salary) AS LowestSalary FROM Employee;
```

Examples of SQL Aggregate Functions

Let's consider a demo **Employee** table to demonstrate SQL aggregate functions . This table contains employee details such as their ID, Name, and Salary.

Id	Name	Salary
1	A	802

Id	Name	Salary
2	B	403
3	C	604
4	D	705
5	E	606
6	F	NULL

1. Count the Total Number of Employees

```
SELECT COUNT(*) AS TotalEmployees FROM Employee;
```

Output:

TotalEmployees
6

2. Calculate the Total Salary

```
SELECT SUM(Salary) AS TotalSalary FROM Employee;
```

Output:

TotalSalary
3120

3. Find the Average Salary:

```
SELECT AVG(Salary) AS AverageSalary FROM Employee;
```

Output:

AverageSalary
624

4. Find the Highest and Lowest Salary

```
SELECT MAX(Salary) AS HighestSalary FROM Employee;
```

Output:

HighestSalary	LowestSalary
802	403

Using Aggregate Functions with GROUP BY

SQL GROUP BY allows us to group rows that have the same values in specific columns. We can then apply aggregate functions to these groups, which helps us summarize data for each group. This is commonly used with the COUNT(), [SUM\(\)](#), AVG(), MIN(), and MAX() functions.

Example: Total Salary by Each Employee

```
SELECT Name, SUM(Salary) AS TotalSalary
FROM Employee
GROUP BY Name;
```

Output:

Name	TotalSalary
A	802
B	403
C	604
D	705
E	606
F	—

Using HAVING with Aggregate Functions

The **HAVING clause** is used to filter results after applying aggregate functions, unlike [WHERE](#), which filters rows before aggregation. [HAVING](#) is essential when we want to filter based on the result of an aggregate function.

Example: Find Employees with Salary Greater Than 600

```
SELECT Name, SUM(Salary) AS TotalSalary
FROM Employee
GROUP BY Name
HAVING SUM(Salary) > 600;
```

Output:

Name	TotalSalary
A	802
C	604

Name	TotalSalary
D	705
E	606

Key Takeaways about SQL Aggregate Functions

- Aggregate functions in SQL operate on a group of values and return a single result.
- They are often used with the GROUP BY clause to summarize the grouped data.
- Aggregate function operates on non-NULL values only (except COUNT).
- Commonly used aggregate functions are – [MIN\(\)](#), [MAX\(\)](#), [COUNT\(\)](#), [AVG\(\)](#), and [SUM\(\)](#).

Conclusion

SQL Aggregate Functions are fundamental tools for summarizing large datasets. Whether you're calculating totals, averages, or finding maximum and minimum values, aggregate functions enable you to quickly perform complex calculations on your data. By mastering these functions, you can gain valuable insights, spot trends, and perform deep analysis on any dataset. With practice, you can leverage these functions with **GROUP BY**, **HAVING**, and other clauses to create powerful and efficient **SQL queries**.

NULL values in SQL

In SQL, some records in a table **may not have values for every field**, and such fields are termed as **NULL values**. These occur when data is unavailable during entry or when the attribute does not apply to a **specific record**. To handle such scenarios, SQL provides a special placeholder value called **NULL** to represent unknown, **unavailable**, or **inapplicable data**.

Importance of NULL Value

It is essential to understand that a **NULL value** differs from a **zero** or an **empty string**. A **NULL** value represents **missing** or **undefined data**. Since it is often not possible to determine which interpretation applies, [SQL](#) treats all **NULL values** as distinct and does not distinguish between them. Typically, it can have one of three interpretations:

1. **Value Unknown:** The value exists but is not known.
2. **Value Not Available:** The value exists but is intentionally withheld.
3. **Attribute Not Applicable:** The value is undefined for a specific record.

Principles of NULL values

- Setting a NULL value is appropriate when the **actual value is unknown**, or when a value is not meaningful.
- A NULL value is not equivalent to a value of **ZERO** if the data type is a number and is not equivalent to spaces if the **data type** is a character.
- A NULL value can be inserted into columns of any data type.
- A NULL value will evaluate NULL in any expression.
- Suppose if any column has a NULL value, then UNIQUE, [FOREIGN](#) key, and CHECK constraints will ignore by SQL.

Logical Behavior

SQL uses **three-valued logic (3VL)**: TRUE, FALSE, and UNKNOWN. Logical expressions involving NULL return UNKNOWN.

- **AND:** Returns FALSE if one operand is FALSE; otherwise, returns UNKNOWN.
- **OR:** Returns TRUE if one operand is TRUE; otherwise, returns UNKNOWN.
- **NOT:** Negates the operand; UNKNOWN remains UNKNOWN.
-

AND	TRUE	FALSE	UNKNOWN
TRUE	TRUE	FALSE	UNKNOWN
FALSE	FALSE	FALSE	FALSE
UNKNOWN	UNKNOWN	FALSE	UNKNOWN

Logical Behaviour of AND

OR	TRUE	FALSE	UNKNOWN
TRUE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	UNKNOWN
UNKNOWN	TRUE	UNKNOWN	UNKNOWN

Logical Behaviour of OR

How To Test for NULL Values?

SQL allows **queries** that check whether an attribute value is **NULL**. Rather than using = or to compare an attribute value to NULL, SQL uses **IS** and **IS NOT**. This is because SQL considers each **NULL value** as being distinct from every other NULL value, so equality comparison is not appropriate.

Example: Employee Table

```
CREATE TABLE Employee (  
    Fname VARCHAR(50),  
    Lname VARCHAR(50),  
    SSN VARCHAR(11),  
    Phoneno VARCHAR(15),  
    Salary FLOAT  
);  
  
INSERT INTO Employee (Fname, Lname, SSN, Phoneno, Salary)  
VALUES  
    ('Shubham', 'Thakur', '123-45-6789', '9876543210', 50000.00),  
    ('Aman', 'Chopra', '234-56-7890', NULL, 45000.00),  
    ('Aditya', 'Arpan', NULL, '8765432109', 55000.00),  
    ('Naveen', 'Patnaik', '345-67-8901', NULL, NULL),  
    ('Nishant', 'Jain', '456-78-9012', '7654321098', 60000.00);  
  
Select * FROM Employee;
```

Output

Fname	Lname	SSN	Phoneno	Salary
Shubham	Thakur	123-45-6789	9876543210	50000
Aman	Chopra	234-56-7890		45000
Aditya	Arpan		8765432109	55000
Naveen	Patnaik	345-67-8901		
Nishant	Jain	456-78-9012	7654321098	60000

Employee Table

The IS NULL Operator

In this query, it retrieves the **Fname** and **Lname** of employees whose **SSN is NULL**. Since SSN represents a **unique identifier**, rows with NULL in this column indicate **missing data**. This query helps **identify records** that lack this **essential information**.

Query:

```
SELECT Fname, Lname FROM Employee WHERE SSN IS NULL;
```

Output

Fname	Lname
Aditya	Arpan

IS NULL Operator

The IS NOT NULL Operator

In this query, it counts the **number of employees** who have a **valid SSN** by excluding rows where SSN is NULL. The result provides the total number of employees with an SSN present in the table. The [COUNT \(*\) function](#) ensures that all **non-NULL rows** are included in the count.

Query

```
SELECT COUNT(*) AS Count FROM Employee WHERE SSN IS NOT NULL;
```

Output

Count
4

IS NOT NULL Operator

Updating NULL Values in a Table

We can update the NULL values present in a table using the **UPDATE** statement in SQL. To do so, we can use the **IS NULL** operator in the [WHERE clause](#) to select the rows with NULL values and then we can set the new value using the [SET keyword](#). Let's suppose that we want to update **SSN** in the row where it is **NULL**.

Query:

```
UPDATE Employee
SET SSN = '789-01-2345'
WHERE Fname = 'Aditya' AND Lname = 'Arpan';
```

```
select* from Employee;
```

Output

Fname	Lname	SSN	Phoneno
Shubham	Thakur	123-45-6789	9876543210
Aman	Chopra	234-56-7890	
Aditya	Arpan	789-01-2345	8765432109
Naveen	Patnaik	345-67-8901	
Nishant	Jain	456-78-9012	7654321098

Updating NULL Values in Table

Explanation:

The UPDATE query modifies the row where the SSN is NULL for the employee named Aditya Arpan, replacing it with a valid value

Conclusion

NULL values in SQL are essential for representing **missing** or **inapplicable** data. Understanding their **behavior**, impact on **constraints**, and three-valued logic is crucial for writing **accurate queries**. By using operators like **IS NULL** and **IS NOT NULL**, we can effectively **filter** and **update** NULL values to maintain [data integrity](#). Mastery of these concepts ensures reliable **data handling** in SQL applications. Mastering these concepts ensures reliable data handling in SQL applications and enhances query precision.

Triggers

In DBMS, triggers are special stored procedures that automatically execute in response to specific database events like INSERT, UPDATE, or DELETE operations on a table or view. They are used to enforce business rules, maintain data integrity, and automate tasks within the database.

Here's a more detailed explanation:

What triggers are:

Stored Procedures:

Triggers are essentially stored procedures, but they are automatically invoked by the database system when a specific event occurs, [as explained by GeoPITS](#).

Event-Driven:

They are associated with a table or view and are triggered by events like data modification (DML statements like INSERT, UPDATE, DELETE) or schema definition (DDL statements like CREATE, ALTER, DROP), [according to GeoPITS](#).

Automatic Execution:

Once the trigger is associated with an event and a table, it executes automatically when that event occurs on that table, [as stated by GeoPITS](#).

What triggers are used for:

Data Integrity:

Triggers can ensure data consistency and accuracy by automatically enforcing constraints or performing validations when data is modified.

Business Rules:

They can enforce business rules that are not easily handled by constraints, [as noted by IBM](#).

Auditing and Logging:

Triggers can record changes made to data, providing an audit trail of database activity.

Automating Tasks:

They can perform tasks like updating other tables or views automatically when a certain event occurs on a specific table.

Data Transformation:

Triggers can also be used to automatically transform or generate values for inserted or updated rows.

Example:

Imagine a table called `employees` with an `salary` column. You could define a trigger that automatically calculates and updates a `total_salaries` column in another table called `company_finance` whenever the `salary` column in `employees` is updated, [as suggested by GeoPITS](#). This would ensure that the `total_salaries` column in `company_finance` always reflects the correct total salary based on changes in the `employees` table.

Trigger in SQL

In this article, you will learn about the trigger and its implementation with examples.

A **Trigger** in Structured Query Language is a set of procedural statements which are executed automatically when there is any response to certain events on the particular table in the database. Triggers are used to protect the data integrity in the database.

In SQL, this concept is the same as the trigger in real life. For example, when we pull the gun trigger, the bullet is fired.

To understand the concept of trigger in SQL, let's take the below hypothetical situation:

Suppose Rishabh is the human resource manager in a multinational company. When the record of a new employee is entered into the database, he has to send the 'Congrats' message to each new employee. If there are four or five employees, Rishabh can do it manually, but if the number of new Employees is more than the thousand, then in such condition, he has to use the trigger in the database.

Thus, now Rishabh has to create the trigger in the table, which will automatically send a 'Congrats' message to the new employees once their record is inserted into the database.

The trigger is always executed with the specific table in the database. If we remove the table, all the triggers associated with that table are also deleted automatically.

In Structured Query Language, triggers are called only either before or after the below events:

1. **INSERT Event:** This event is called when the new row is entered in the table.
2. **UPDATE Event:** This event is called when the existing record is changed or modified in the table.
3. **DELETE Event:** This event is called when the existing record is removed from the table.

Types of Triggers in SQL

Following are the six types of triggers in SQL:

- | | | |
|---|---------------|----------------|
| 1. AFTER | INSERT | Trigger |
| This trigger is invoked after the insertion of data in the table. | | |
| 2. AFTER | UPDATE | Trigger |
| This trigger is invoked in SQL after the modification of the data in the table. | | |
| 3. AFTER | DELETE | Trigger |
| This trigger is invoked after deleting the data from the table. | | |
| 4. BEFORE | INSERT | Trigger |
| This trigger is invoked before the inserting the record in the table. | | |
| 5. BEFORE | UPDATE | Trigger |
| This trigger is invoked before the updating the record in the table. | | |
| 6. BEFORE | DELETE | Trigger |
| This trigger is invoked before deleting the record from the table. | | |

Syntax of Trigger in SQL

1. **CREATE TRIGGER** Trigger_Name
2. [BEFORE | **AFTER**] [**Insert** | **Update** | **Delete**]
3. **ON** [Table_Name]
4. [**FOR EACH ROW** | **FOR EACH COLUMN**]
5. **AS**
6. **Set of** SQL Statement

In the trigger syntax, firstly, we have to define the name of the trigger after the CREATE TRIGGER keyword. After that, we have to define the BEFORE or AFTER keyword with anyone event.

Then, we define the name of that table on which trigger is to occur.

After the table name, we have to define the row-level or statement-level trigger.

And, at last, we have to write the SQL statements which perform actions on the occurring of event.

Example of Trigger in SQL

To understand the concept of trigger in SQL, first, we have to create the table on which trigger is to be executed.

Field	Type	NULL	Key	Default	Extra
Student_RollNo	INT	NO	PRI	NULL	
Student_FirstName	Varchar(100)	YES		NULL	
Student_EnglishMarks	INT	YES		NULL	
Student_PhysicsMarks	INT	YES		NULL	
Student_ChemistryMarks	INT	YES		NULL	
Student_MathsMarks	INT	YES		NULL	
Student_TotalMarks	INT	YES		NULL	
Student_Percentage	INT	YES		NULL	

The following query creates the **Student_Trigger** table in the SQL database:

1. **CREATE TABLE** Student_Trigger
2. (
3. Student_RollNo **INT** NOT NULL **PRIMARY KEY**,
4. Student_FirstName **Varchar** (100),

5. Student_EnglishMarks **INT**,
6. Student_PhysicsMarks **INT**,
7. Student_ChemistryMarks **INT**,
8. Student_MathsMarks **INT**,
9. Student_TotalMarks **INT**,
10. Student_Percentage);

The following query shows the structure of the **Student_Trigger** table:

1. **DESC** Student_Trigger;

Output:

The following query fires a trigger before the insertion of the student record in the table:

SQL Stored Procedures

An SQL **stored procedure** is a group of pre-compiled SQL statements (prepared SQL code) that can be reused by simply calling it whenever needed.

It can be used to perform a wide range of database operations such as inserting, updating, or deleting data, generating reports, and performing complex calculations. Stored procedures are very useful because they allow you to encapsulate (bundle) a set of SQL statements as a single unit and execute them repeatedly with different parameters, making it easy to manage and reuse the code.

Syntax

The basic syntax to create an SQL stored procedure is as follows –

```
DELIMITER //
CREATE PROCEDURE procedure_name(parameter1 datatype, parameter2 datatype, ...)
BEGIN
    -- SQL statements to be executed
END
DELIMITER ;
```

- The CREATE PROCEDURE statement is used to create the procedure. We can define any number of input parameters as per the requirement.
- The SQL statements that make up the procedure are placed between the BEGIN and END keywords.

Creating a Procedure

We can create a stored procedure using the CREATE PROCEDURE statement in SQL. Following are the simple steps for creating a stored procedure –

- Choose a name for the procedure.
- Write the SQL code for the procedure.
- We can then test the stored procedure by executing it with different input parameters.

Example

To understand it better let us consider the CUSTOMERS table which contains the personal details of customers including their name, age, address and salary etc. as shown below –

```
CREATE TABLE CUSTOMERS (  
  ID INT NOT NULL,  
  NAME VARCHAR (20) NOT NULL,  
  AGE INT NOT NULL,  
  ADDRESS CHAR (25),  
  SALARY DECIMAL (18, 2),  
  PRIMARY KEY (ID)  
);
```

Now, insert values into this table using the INSERT statement as follows –

```
INSERT INTO CUSTOMERS VALUES  
(1, 'Ramesh', 32, 'Ahmedabad', 2000.00),  
(2, 'Khilan', 25, 'Delhi', 1500.00),  
(3, 'Kaushik', 23, 'Kota', 2000.00),  
(4, 'Chaitali', 25, 'Mumbai', 6500.00),  
(5, 'Hardik', 27, 'Bhopal', 8500.00),  
(6, 'Komal', 22, 'Hyderabad', 4500.00),  
(7, 'Muffy', 24, 'Indore', 10000.00);
```

The table will be created as –

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	Kaushik	23	Kota	2000.00

4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	Hyderabad	4500.00
7	Muffy	24	Indore	10000.00

Now, let us look at a simple example of creating a stored procedure that takes an input parameter and returns a result set.

In the following query, we are creating the stored procedure with the name GetCustomerInfo. then we provide it with a single input parameter called @CutomerAge. The stored procedure then selects all records from the CUSTOMERS table where the value of the CutomerAge matches the input parameter.

```
DELIMITER //
CREATE PROCEDURE GetCustomerInfo(IN CustomerAge INT)
BEGIN
    SELECT * FROM CUSTOMERS WHERE AGE = CustomerAge;
END //
DELIMITER ;
```

Output

This would produce the following result –

Query OK, 0 rows affected (0.01 sec)

Verification

We can test the stored procedure by executing it using the **CALL** statement as shown below –

```
CALL GetCustomerInfo(25);
```

This will return all columns from the CUSTOMERS table where the customers age is 25.

ID	NAME	AGE	ADDRESS	SALARY
2	Khilan	25	Delhi	1500.00
4	Chaitali	25	Mumbai	6500.00

Stored Procedure Parameter Types

Stored procedures in a database system can have different types of parameters, which are placeholders for values that will be passed to the stored procedure when it is executed. Following are the different types of stored procedure parameters in SQL –

S.No.	Parameter & Description
1	Input parameters These parameters are used to pass values from the calling statement to the stored procedure.
2	Output parameters These parameters are used to return values from the stored procedure.
3	Input/Output parameters These parameters allow a stored procedure to accept input values and return output values.

Procedure with IN parameter

IN is the default parameter of the procedure that will receive input values. We can pass the values as arguments when the stored procedure is being called.

These values are read-only, so they cannot be modified by the stored procedure.

Example

In the following query, we are creating a stored procedure that takes a customer ID as an input parameter and returns the corresponding customer salary.

The procedure body simply performs a SELECT statement to retrieve the "Salary" column from the "CUSTOMERS" table, where the "CustomerID" matches the input parameter.

```
DELIMITER //  
CREATE PROCEDURE GetCustomerSalary(IN CustomerID Int)  
BEGIN  
    SELECT SALARY FROM CUSTOMERS WHERE ID = CustomerID;  
END //  
DELIMITER ;
```

Output

This would produce the following result –

Query OK, 0 rows affected (0.01 sec)

Verification

We can test it by executing it with different ID as an input parameter as shown in the query below –

```
CALL GetCustomerSalary(6);
```

This will return the salary for the customer with an ID of 6, assuming there is a corresponding row in the CUSTOMERS table –

SALARY
4500.00

Procedure with OUT parameter

The OUT parameter is used to return the output value from the procedure.

Note that when using an OUT parameter, we must specify the keyword OUT before the parameter name when passing it to the stored procedure. This tells the SQL database that the parameter is an output parameter and should be assigned with a value in the stored procedure.

Example

In the following query we are creating a stored procedure that used to count the number of records of customer having same age and assign this count to the 'total' variable which holds the number of records.

The procedure body performs a SELECT statement to get the count of records having same age from the "CUSTOMERS" table

```
DELIMITER //
CREATE PROCEDURE GetDetail(OUT total INT)
BEGIN
    SELECT COUNT(AGE) INTO total FROM CUSTOMERS
    WHERE AGE = 25;
END //
DELIMITER ;
```

Calling the created procedure and passing the 'total' parameter

```
CALL GetDetail(@total);
```

Here, we are using the SELECT statement and getting the count –

```
SELECT @total;
```

Output

This would produce the following result –

@total
2

Verification

To verify whether the procedure is created, we can use the following query –

```
SHOW CREATE PROCEDURE GetDetails;
```

Procedure with INOUT parameter

The INOUT parameter is a combination of an IN parameter and an OUT parameter. You can pass data into the stored procedure and receive data from the stored procedure using the same parameter.

To declare an INOUT parameter in a stored procedure, we need to specify the **INOUT** keyword before the parameter name.

Example

In the following query, we provide two INOUT parameters to the stored procedure: **cust_id** and **curr_Salary**. These two are used as both an input and output parameters.

The stored procedure first retrieves the current salary of the customer from the database using the **cust_id** parameter. It then increases the salary by 10% and updates the customer's salary in the database using the same parameter.

```
DELIMITER //
CREATE PROCEDURE increaseSalary(INOUT Cust_Id Int, INOUT curr_Salary Int)
BEGIN
    SELECT SALARY INTO curr_Salary FROM CUSTOMERS Where ID = Cust_Id;
    SET curr_Salary = curr_Salary * 1.1;
    Update CUSTOMERS SET SALARY = curr_Salary Where ID = Cust_Id;
END //
DELIMITER ;
```

Output

This would produce the following result –

Query OK, 0 rows affected (0.01 sec)

Verification

We can test it by executing it with different ID or input parameters as shown in the query below –

```
SET @customerID = 1;  
SET @salary = 0.0;  
CALL increaseSalary(@customerID, @salary);
```

Following is Query to select the updated salary from the stored procedure

```
SELECT @salary AS updated_salary;
```

The result-set is obtained as –

updated_salary

2200

Advantages of Stored Procedures

Following are the advantages of stored procedures –

- **Improved Performance:** Stored procedures are pre-compiled and stored on the server, so they can be executed more quickly than SQL statements that are sent from client applications.
- **Code Reuse:** Stored procedures can be called from different client applications, which means that the same code can be reused across different applications. This reduces development time and maintenance costs.
- **Reduced Network Traffic:** Because stored procedures are executed on the server, only the results are returned to the client, which reduces network traffic and improves application performance.
- **Better Security:** Stored procedures can be used to enforce security rules and prevent unauthorized access to sensitive data. They can also limit the actions that can be performed by users, making it easier to maintain data integrity and consistency.
- **Simplified Maintenance:** By storing SQL code in a single location, it becomes easier to maintain and update the code. This makes it easier to fix bugs, add new functionality, and optimize performance.

Drawbacks of Stored Procedures

Following are the disadvantages of stored procedures –

- **Increased Overhead:** Stored procedures can consume more server resources than simple SQL statements, particularly when they are used frequently or for complex operations.
- **Limited Portability:** Stored procedures are often specific to a particular database management system (DBMS), which means they may not be easily portable to other DBMSs.
- **Debugging Challenges:** Debugging stored procedures can be more challenging than debugging simple SQL statements, particularly when there are multiple layers of code involved.
- **Security Risks:** If stored procedures are not written correctly, they can pose a security risk, particularly if they are used to access sensitive data or to perform actions that could compromise the integrity of the database.

In DBMS, a stored procedure is a named set of SQL statements stored as a unit within the database. It's essentially a pre-compiled block of code that can be executed repeatedly with a single call, offering benefits like enhanced performance, security, and code reusability.

Key aspects of stored procedures:

Definition:

A stored procedure is a pre-compiled block of SQL code stored within the database.

Purpose:

They are designed to encapsulate frequently used SQL logic, reducing redundancy and improving application efficiency.

Advantages:

Performance: Stored procedures can be faster than executing individual SQL statements due to pre-compilation.

Security: They allow you to control access to the database at a procedure level, limiting the exposure of underlying tables.

Code Reusability: They can be reused across multiple applications and programs.

Modularity: They promote modular code design, making it easier to maintain and debug.

Types (SQL Server Example):

System-defined stored procedures: These are pre-defined procedures provided by the database system for administrative tasks.

User-defined stored procedures: These are custom-created procedures by database users.

Extended stored procedures: These offer extended functionality beyond standard SQL.

CLR (Common Language Runtime) stored procedures: These allow using .NET languages (C#, VB.NET) to create stored procedures, [according to ScholarHat](#).

Execution:

Stored procedures are invoked by name, similar to a function call.

Parameters:

They can accept input parameters, allowing for dynamic behavior based on the values passed in.
They can also return output values.

Creation and Deletion:

Stored procedures are created using the `CREATE PROCEDURE` statement and can be dropped using the `DROP PROCEDURE` statement.

Nested subqueries

Nested subqueries in database management systems (DBMS) are SQL queries embedded within other SQL queries. They are also known as inner queries or nested queries. Essentially, a subquery is a query within another query, allowing the results of one query to be used as input for a subsequent query.

Here's a more detailed explanation:

What are Nested Subqueries?

- A subquery is a query that is nested inside another query.
- The inner query (subquery) is executed first, and its results are then used by the outer query.
- Subqueries can be used in various parts of a SQL statement, including `SELECT`, `FROM`, `WHERE`, `HAVING`, and `INSERT` clauses, [according to DbVisualizer](#).
- Nested subqueries can reference columns from the outer query, [according to StudySmarter UK](#).

Why Use Nested Subqueries?

Complexity:

They allow you to perform complex data retrieval and manipulation scenarios by breaking down operations into smaller, manageable steps.

Data Filtering:

You can filter data based on the results of another query, making it possible to find specific rows based on conditions defined by a different query, according to Medium.

Data Transformation:

Nested subqueries can be used to transform data, for example, by calculating derived values or performing aggregations within a subquery and then using those values in the main query.

Simplified Code:

They can help simplify complex SQL code by breaking down complex operations into smaller, more manageable parts.

Example:

Let's say you have two tables: `employees` and `departments`. You want to find all employees who work in departments with more than 10 employees. You can use a nested subquery:

Code

```
SELECT *  
FROM employees  
WHERE department_id IN (SELECT department_id  
                        FROM departments  
                        WHERE department_id IN (SELECT department_id FROM  
employees GROUP BY department_id HAVING COUNT(*) > 10));
```



In this example, the inner query selects the `department_id` of departments with more than 10 employees. The outer query then uses this list of `department_id` values to find all employees who belong to those departments.

Views in SQL

- Views in SQL are considered as a virtual table. A view also contains rows and columns.
- To create the view, we can select the fields from one or more tables present in the database.
- A view can either have specific rows based on certain condition or all the rows of a table.

Advantages of View:

1. **Complexity:** Views help to reduce the complexity. Different views can be created on the same base table for different users.
2. **Security:** It increases the security by excluding the sensitive information from the view.
3. **Query Simplicity:** It helps to simplify commands from the user. A view can draw data from several different tables and present it as a single table.
4. **Consistency:** A view can present a consistent, unchanged image of the structure of the database. Views can be used to rename the columns without affecting the base table.
5. **Data Integrity:** If data is accessed and entered through a view, the DBMS can automatically check the data to ensure that it meets the specified integrity constraints.
6. **Storage Capacity:** Views take very little space to store the data.
7. **Logical Data Independence:** View can make the application and database tables to a certain extent independent.

Disadvantages of View:

The DML statements which can be performed on a view created using single base table have certain restrictions are:

1. You cannot INSERT if the base table has any not null column that do not appear in view.
2. You cannot INSERT or UPDATE if any of the column referenced in the INSERT or UPDATE contains group functions or columns defined by expression.
3. You can't execute INSERT, UPDATE, DELETE statements on a view if with read only option is enabled.
4. You can't be created view on temporary tables.
5. You cannot INSERT, UPDATE, DELETE if the view contains group functions GROUP BY, DISTINCT or a reference to a pseudocolumn rownum.
6. You can't pass parameters to the SQL server views.
7. You can't associate rules and defaults with views.

Sample table:

Student_Detail

STU_ID	NAME	ADDRESS
1	Stephan	Delhi

2	Kathrin	Noida
3	David	Ghaziabad
4	Alina	Gurugram

Student_Marks

STU_ID	NAME	MARKS	AGE
1	Stephan	97	19
2	Kathrin	86	21
3	David	74	18
4	Alina	90	20
5	John	96	18

1. Creating view

A view can be created using the **CREATE VIEW** statement. We can create a view from a single table or multiple tables.

Syntax:

1. **CREATE VIEW** view_name **AS**
2. **SELECT** column1, column2.....
3. **FROM** table_name
4. **WHERE** condition;

2. Creating View from a single table

In this example, we create a View named DetailsView from the table Student_Detail.

Query:

1. **CREATE VIEW** DetailsView **AS**
2. **SELECT NAME, ADDRESS**
3. **FROM** Student_Details
4. **WHERE** STU_ID < 4;

Just like table query, we can query the view to view the data.

1. **SELECT * FROM** DetailsView;

Output:

NAME	ADDRESS
Stephan	Delhi
Kathrin	Noida
David	Ghaziabad

3. Creating View from multiple tables

View from multiple tables can be created by simply include multiple tables in the SELECT statement.

In the given example, a view is created named MarksView from two tables Student_Detail and Student_Marks.

Query:

1. **CREATE VIEW** MarksView **AS**
2. **SELECT** Student_Detail.**NAME**, Student_Detail.ADDRESS, Student_Marks.MARKS
3. **FROM** Student_Detail, Student_Mark
4. **WHERE** Student_Detail.**NAME** = Student_Marks.**NAME**;

To display data of View MarksView:

1. **SELECT * FROM** MarksView;

NAME	ADDRESS	MARKS
Stephan	Delhi	97
Kathrin	Noida	86
David	Ghaziabad	74
Alina	Gurugram	90

4. Deleting View

A view can be deleted using the Drop View statement.

Syntax

1. **DROP VIEW** view_name;

Example:

If we want to delete the View **MarksView**, we can do this as:

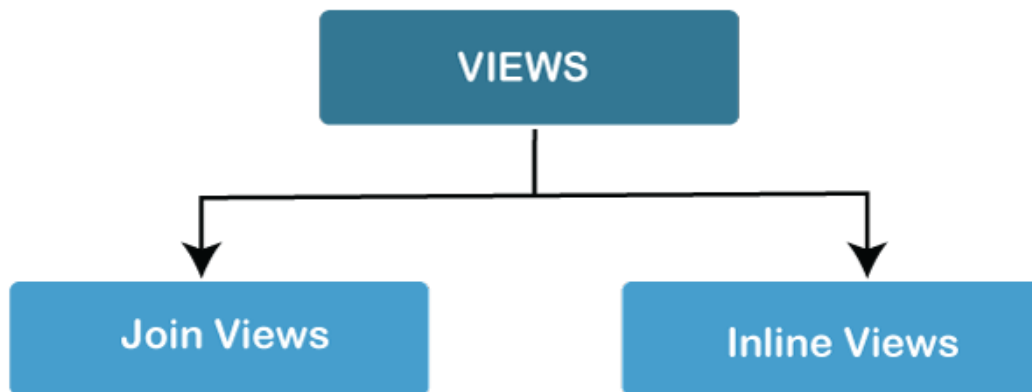
1. **DROP VIEW** MarksView;

Significance of Views:

Views are highly significant, as they can provide advantages over tasks. Views can represent a subset of data contained in a table. Consequently they can limit the degree of exposure of the underlying base table to the outer world. They are used for security purpose in database and act as an intermediate between real table schemas and programmability. They act as aggregate tables.

Types of Views:

There are two types of views.



1. **Join View:** A join view is a view that has more than one table or view in its from clause and it does not use any Group by Clause, Rownum, Distinct and set operation.
2. **Inline View:** An inline view is a view which is created by replacing a subquery in the from clause which defines the data source that can be referenced in the main query. The sub query must be given an alias for efficient working.

Assertion

In DBMS (Database Management System), an assertion is a statement that specifies a condition that must always be true for the data in the database. It's used to enforce business rules and ensure data integrity. Essentially, it's a constraint that checks if a specific condition is met before allowing an operation (like inserting or updating data) to proceed.

Key characteristics of an assertion:

Predicate:

An assertion is a predicate (a statement that can be true or false) that must evaluate to true for the database to maintain its integrity.

Enforcement:

If an assertion's condition is violated, the database typically rolls back the transaction and prevents the operation from completing, [says reintechnology.com](#).

Scope:

Assertions can apply to one or more tables and attributes, offering more flexibility than simple constraints that usually operate on a single table or attribute, [says The University of Virginia](#).

Example:

An assertion might state that the total amount of orders in the `Orders` table must always be less than 10,000.

How Assertions Differ from Constraints:

Scope:

Assertions can encompass multiple tables and attributes, whereas constraints typically focus on a single table and attribute, [says The University of Virginia](#).

Complexity:

Assertions allow for more complex conditions to be expressed and enforced than simple constraints, [says The University of Virginia](#).

Enforcement:

Constraints are typically enforced at the table level, while assertions can be checked more broadly across the database, [says The University of Virginia](#).

When to Use Assertions:

- Enforcing complex business rules that span multiple tables or attributes.
- Ensuring data consistency and integrity across the database.
- Creating assertions for data quality testing, as used in tools like Dataform.

Control methods of Database Security

Database Security means keeping sensitive information safe and prevent the loss of data. Security of data base is controlled by Database Administrator (DBA).

The following are the main control measures are used to provide security of data in databases:

1. Authentication
2. Access control
3. Inference control
4. Flow control
5. Database Security applying Statistical Method
6. Encryption

These are explained as following below.

1. **Authentication :**

Authentication is the process of confirmation that whether the user log in only according to the rights provided to him to perform the activities of data base. A particular user can login only up to his privilege but he can't access the other sensitive data. The privilege of accessing sensitive data is restricted by using Authentication.

By using these authentication tools for biometrics such as retina and figure prints can prevent the data base from unauthorized/malicious users.

2. **Access Control :**

The security mechanism of DBMS must include some provisions for restricting access to the data base by unauthorized users. Access control is done by creating user accounts and to control login process by the DBMS. So, that database access of sensitive data is possible only to those people (database users) who are allowed to access such data and to restrict access to unauthorized persons.

The database system must also keep the track of all operations performed by certain user throughout the entire login time.

3. **Inference Control :**

This method is known as the countermeasures to statistical database security problem. It is used to prevent the user from completing any inference channel. This method protect sensitive information from indirect disclosure.

Inferences are of two types, identity disclosure or attribute disclosure.

4. **Flow Control :**

This prevents information from flowing in a way that it reaches unauthorized users. Channels are the pathways for information to flow implicitly in ways that violate the privacy policy of a company are called covert channels.

5. **Database Security applying Statistical Method :**

Statistical database security focuses on the protection of confidential individual values stored in and used for statistical purposes and used to retrieve the summaries of values based on categories. They do not permit to retrieve the individual information.

This allows to access the database to get statistical information about the number of employees in the company but not to access the detailed confidential/personal information about the specific individual employee.

6. **Encryption :**

This method is mainly used to protect sensitive data (such as credit card numbers, OTP numbers) and other sensitive numbers. The data is encoded using some encoding algorithms.

An unauthorized user who tries to access this encoded data will face difficulty in decoding it, but authorized users are given decoding keys to decode data.

PL/SQL Introduction

PL/SQL (Procedural Language/Structured Query Language) is a **block-structured language** developed by **Oracle** that allows developers to combine the power of **SQL** with procedural programming constructs. The PL/SQL language enables efficient data manipulation and control-flow logic, all within the **Oracle Database**.

In this article, we'll cover **PL/SQL basics**, including its core features, PL/SQL block structure, and practical examples that demonstrate the power of PL/SQL. We'll also explore the differences between **SQL** and PL/SQL, how variables and identifiers work, and how the PL/SQL execution environment operates within Oracle.

Basics of PL/SQL

- PL/SQL stands for Procedural Language extensions to the Structured Query Language (SQL).
- PL/SQL is a combination of SQL along with the procedural features of programming languages.
- Oracle uses a PL/SQL engine to process the PL/SQL statements.
- PL/SQL includes procedural language elements like conditions and loops. It allows declaration of constants and variables, procedures and functions, types and variable of those types and triggers.

Features of PL/SQL

1. PL/SQL is basically a procedural language, which provides the functionality of decision-making, iteration, and many more features of procedural programming languages.
2. PL/SQL can execute a number of queries in one block using single command.
3. One can create a PL/SQL unit such as procedures, functions, packages, triggers, and types, which are stored in the database for reuse by applications.
4. PL/SQL provides a feature to handle the exception which occurs in PL/SQL block known as exception handling block.
5. Applications written in PL/SQL are portable to computer hardware or operating system where Oracle is operational.
6. PL/SQL Offers extensive error checking.
- 7.

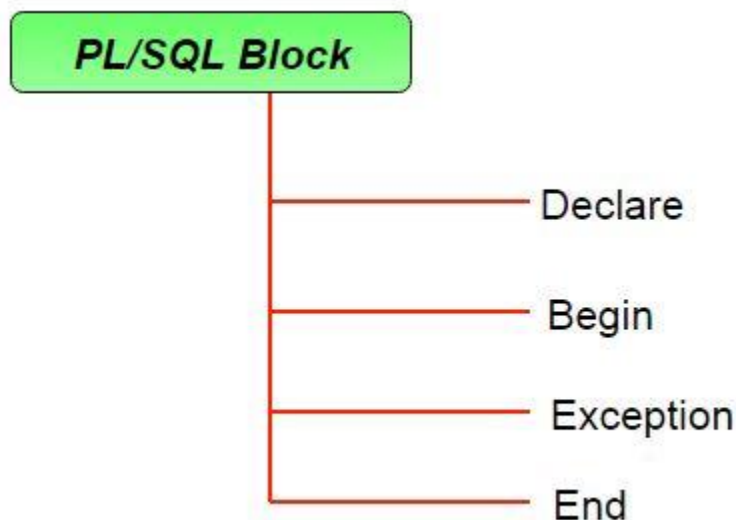
Differences Between SQL and PL/SQL

SQL	PL/SQL
SQL is a single query that is used to perform DML and DDL operations.	PL/SQL is a block of codes that used to write the entire program blocks/ procedure/ function, etc.
It is declarative, that defines what needs to be done, rather than how things need to be done.	PL/SQL is procedural that defines how the things needs to be done.
Execute as a single statement.	Execute as a whole block.
Mainly used to manipulate data.	Mainly used to create an application.

SQL	PL/SQL
Cannot contain PL/SQL code in it.	It is an extension of SQL, so it can contain SQL inside it.

Structure of PL/SQL Block

PL/SQL extends [SQL](#) by adding constructs found in **procedural languages**, resulting in a structural language that is more powerful than SQL. The basic unit in PL/SQL is a block. All PL/SQL programs are made up of blocks, which can be nested within each other.



Typically, each block performs a logical action in the program. A block has the following structure:

```

DECLARE
    declaration statements;

BEGIN
    executable statements

EXCEPTIONS
    exception handling statements

END;
  
```

- Declare section starts with **DECLARE** keyword in which variables, constants, records as cursors can be declared which stores data temporarily. It basically consists definition of PL/SQL identifiers. This part of the code is optional.
- Execution section starts with **BEGIN** and ends with **END** keyword. This is a mandatory section and here the program logic is written to perform any task like loops and conditional statements. It supports all [DML](#) commands, [DDL](#) commands and SQL*PLUS built-in functions as well.
- Exception section starts with **EXCEPTION** keyword. This section is optional which contains statements that are executed when a run-time error occurs. Any exceptions can be handled in this section.

PL/SQL Identifiers

There are several PL/SQL identifiers such as variables, constants, procedures, cursors, triggers etc.

1. **Variables:** Like several other programming languages, variables in PL/SQL must be declared prior to its use. They should have a valid name and data type as well. Syntax for declaration of variables:

```
variable_name datatype [NOT NULL := value ];
```

1. Example to show how to declare variables in PL/SQL :

```
SQL> SET SERVEROUTPUT ON;
```

```
SQL> DECLARE
    var1 INTEGER;
    var2 REAL;
    var3 varchar2(20) ;
```

```
BEGIN
    null;
END;
/
```

1. Output:

```
PL/SQL procedure successfully completed.
```

1. **Explanation:**

- **SET SERVEROUTPUT ON:** It is used to display the buffer used by the dbms_output.
- **var1 INTEGER :** It is the declaration of variable, named **var1** which is of integer type. There are many other data types that can be used like float, int, real, smallint, long etc. It also supports variables used in SQL as well like NUMBER(prec, scale), varchar, varchar2 etc.
- **PL/SQL procedure successfully completed.:** It is displayed when the code is compiled and executed successfully.
- **Slash (/) after END;:** The slash (/) tells the SQL*Plus to execute the block.
- **Assignment operator (:=) :** It is used to assign a value to a variable.

2. **Displaying Output:** The outputs are displayed by using DBMS_OUTPUT which is a built-in package that enables the user to display output, debugging information, and send messages from PL/SQL blocks, subprograms, packages, and triggers. Let us see an example to see how to display a message using PL/SQL :

```
SQL> SET SERVEROUTPUT ON;
SQL> DECLARE
    var varchar2(40) := 'I love GeeksForGeeks' ;

    BEGIN
        dbms_output.put_line(var);

    END;
/
```

1. Output:

```
I love GeeksForGeeks
```

```
PL/SQL procedure successfully completed.
```

1. **Explanation:**

- **dbms_output.put_line :** This command is used to direct the PL/SQL output to a screen.

2. **Using Comments:** Like in many other programming languages, in PL/SQL also, comments can be put within the code which has no effect in the code. There are two syntaxes to create comments in PL/SQL :
 - **Single Line Comment:** To create a single line comment , the symbol -- is used.
 - **Multi Line Comment:** To create comments that span over several lines, the symbol /* and */ is used.
3. **Taking input from user:** Just like in other programming languages, in PL/SQL also, we can take input from the user and store it in a variable. Let us see an example to show how to take input from users in PL/SQL:

```
SQL> SET SERVEROUTPUT ON;

SQL> DECLARE

    -- taking input for variable a
    a number := &a;

    -- taking input for variable b
    b varchar2(30) := &b;

BEGIN
    null;

END;
/
```

1. Output:

```
Enter value for a: 24
old   2: a number := &a;
new   2: a number := 24;
Enter value for b: 'GeeksForGeeks'
old   3: b varchar2(30) := &b;
new   3: b varchar2(30) := 'GeeksForGeeks';
```

PL/SQL procedure successfully completed.

1. **(***) Let us see an example on PL/SQL to demonstrate all above concepts in one single block of code.**

--PL/SQL code to print sum of two numbers taken from the user.

```
SQL> SET SERVEROUTPUT ON;

SQL> DECLARE

    -- taking input for variable a
    a integer := &a ;

    -- taking input for variable b
    b integer := &b ;
    c integer ;

BEGIN
    c := a + b ;
```

```
        dbms_output.put_line('Sum of '||a||' and '||b||' is = '||c);  
  
    END;  
    /  
Enter value for a: 2  
Enter value for b: 3  
  
Sum of 2 and 3 is = 5  
  
PL/SQL procedure successfully completed.
```

PL/SQL Execution Environment:

The PL/SQL engine resides in the Oracle engine. The Oracle engine can process not only single SQL statement but also block of many statements. The call to Oracle engine needs to be made only once to execute any number of SQL statements if these SQL statements are bundled inside a PL/SQL block.